Homework 04 T-tests, Sampling Distributions, and the Bootstrap

Due by 11:59pm, Tuesday 2.17.26

S&DS 2300/5300/ENV 757

(1) Practice with Loops. *(10 points)* For this problem, use loops even if you could do the task without them.

(1.1) A Tribonacci sequence is a series of integers where each number after the third number is found by adding together the three integers before it. Starting with 0, 1, 1, the sequence goes:

> 0, 1, 1, 2, 4, 7, ...

Write a loop that fills a vector called `myTrib` with this sequence going up to a total length of 21 numbers. Display the final value of `myTrib` AND write a line of code that calculates how many digits are in the final number in `myTrib`.

(1.2) Here is the link to the World Bank data from 2024:

> https://raw.githubusercontent.com/jreuning/sds230_data/refs/heads/main/WB_2024.csv

Read the data into a dataframe called `wb`. Write a loop to fill a vector called `compVals` having length equal to the number of columns in the World Bank data frame. The i-th entry in `compVals` should be a number (>= 0) equal to the total number of non-missing values in the i-th column of World Bank data frame. Make a histogram of `compVals` and label as appropriate. In addition, make a histogram that displays the units on the horizonal axis in terms of percentages (not proportions) for each variable (rather than totals). Write a sentence or two about what these plots tell you about the dataset.

(For full credit, use only one for-loop to do part 1.2)

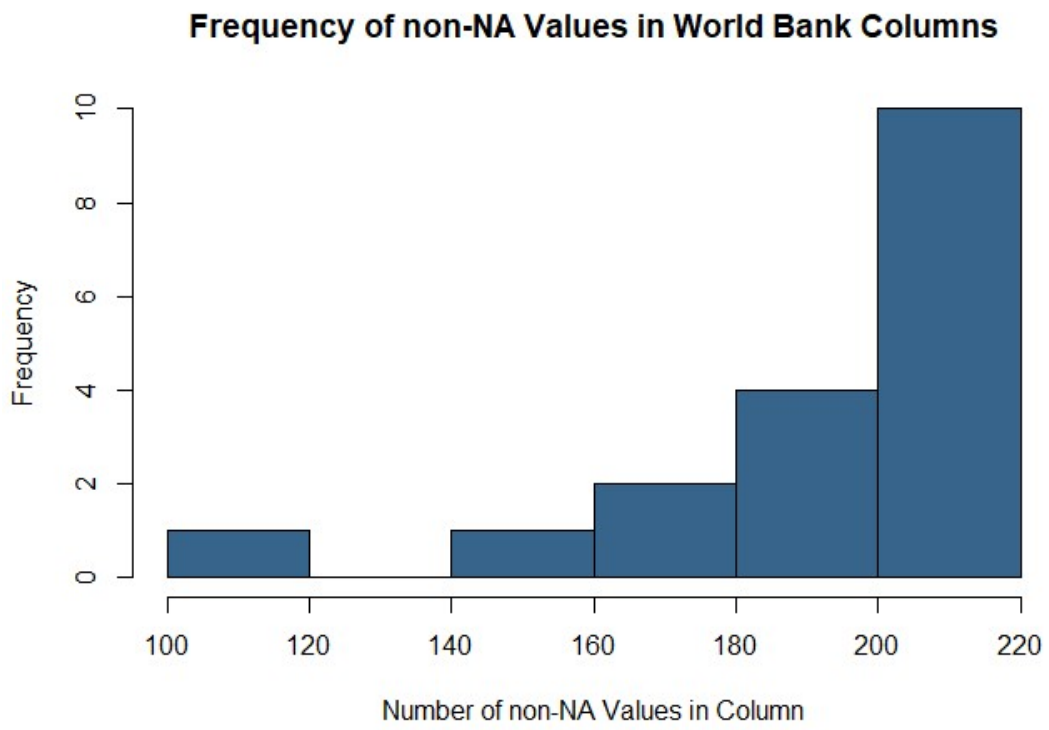
```
# 1.1 create Tribonacci sequence
myTrib <- rep(NA, 21)
myTrib[1:3] <- c(0, 1, 1)

for (i in 4:21){
  myTrib[i] <- myTrib[i-1] + myTrib[i-2] + myTrib[i-3]
}
print(myTrib)
```

```
##  [1]     0     1     1     2     4     7    13    24    44    81   149
274
## [13]   504   927  1705  3136  5768 10609 19513 35890 66012
```

```r
# calculates digits in final number
nchar(myTrib[21])
```

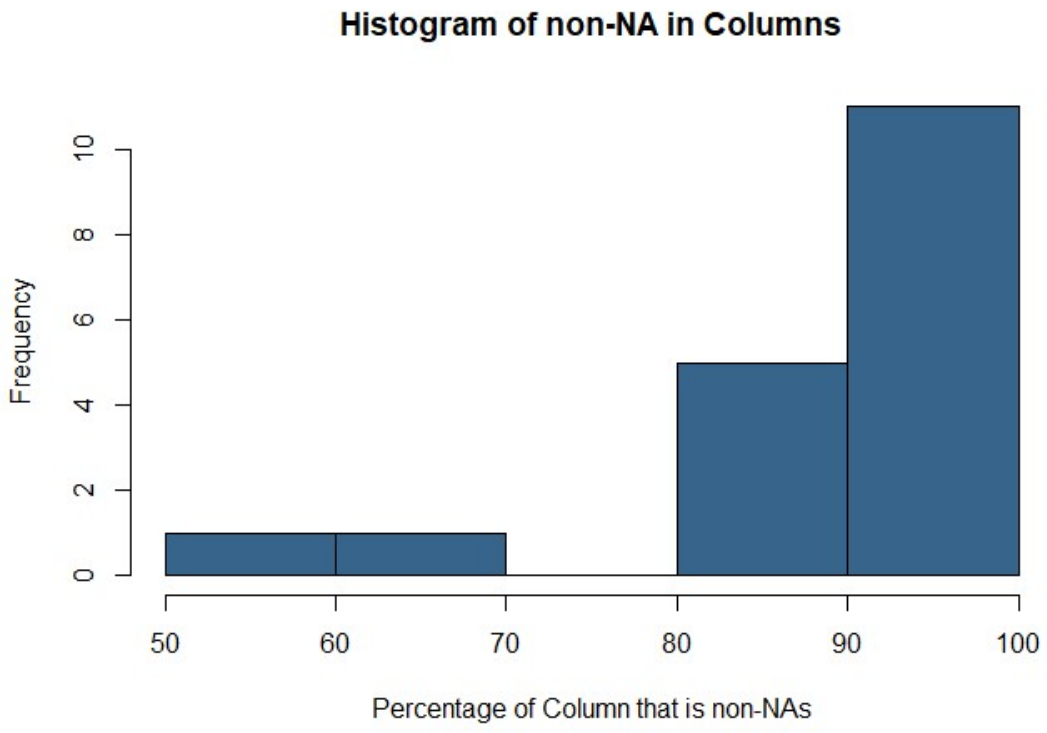
```
## [1] 5
```

```r
# 1.2
wb <-
read.csv("https://raw.githubusercontent.com/jreuning/sds230_data/refs/heads/main/WB_2024.csv")

compVals <- rep(NA, ncol(wb))

for (i in seq_len(ncol(wb))) {
  compVals[i] <- sum(!is.na(wb[, i]))
}

# histogram of compVals
hist(compVals,
     main = "Frequency of non-NA Values in World Bank Columns",
     col = "steelblue4",
     xlab = "Number of non-NA Values in Column")
```




```r
# create percentage labels
compVals_percents <- round(compVals/nrow(wb)*100)

# additional histogram
```

```r
hist(compVals_percents,
     main = "Histogram of non-NA in Columns",
     col = "steelblue4",
     xlab = "Percentage of Column that is non-NAs")
```



Both histograms tell us that there are a lot of columns with complete data. The first histogram tells us the raw value distribution but does not provide any info on how many columns there are in the whole dataset, whereas the second histogram tells us that most of the columns have almost all of their columns completed and others have only 50% complete data.

(2) Simulations with the Exponential Distribution *(40 points).*

For this problem, we'll investigate the sampling distributional characteristics of three statistics. In particular, suppose we take a sample of size 20 from an exponential distribution. We can use the CLT to say something about how far the sample mean is likely to be from the true mean, but how far are the sample median or the sample variance likely to be from the true values in an exponential distribution where we take a sample of size 20? Also, what do we expect the distribution of these statistics to look like?

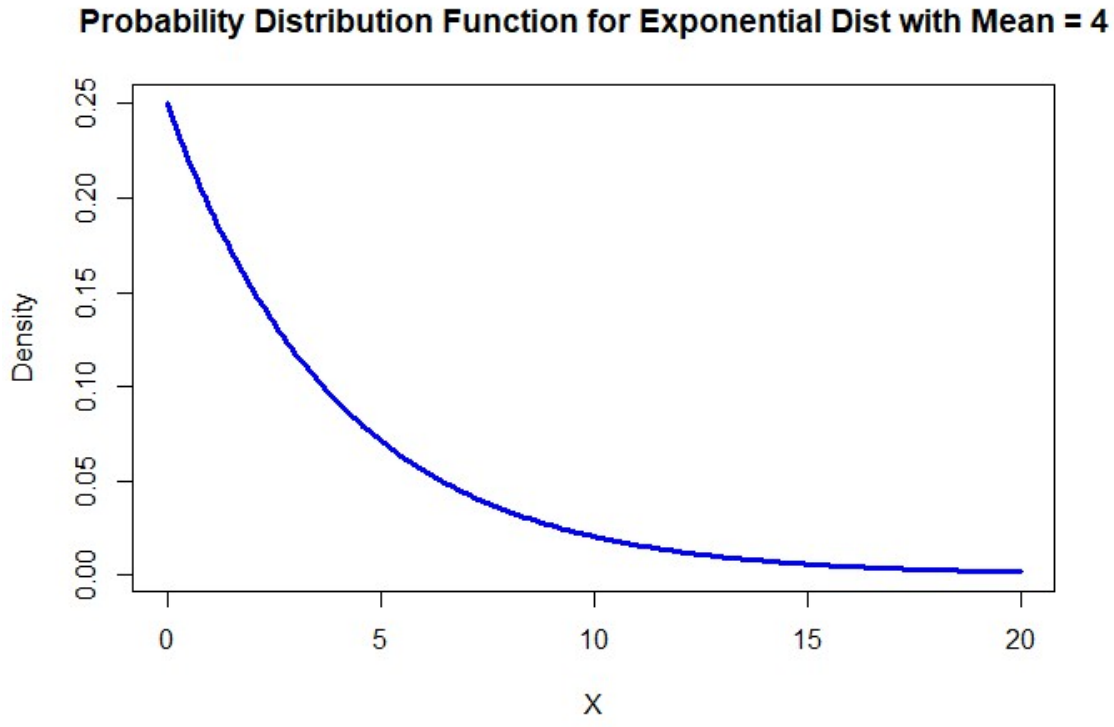
(2.1) *(10 points)* First, let's get a quick sense of what an exponential distribution looks like where the mean is 4. By the way, it's handy to know that for an exponential distribution with mean 4, the variance is 16 and the median is 4*ln(2). You can read about the exponential distribution HERE.

The code below gives a quick plot of this distribution. Your job is to succinctly answer what each part of the code does. You'll probably need to get help on the function `dexp()`, `seq()` and on a few of the graphics parameters in `par()`.

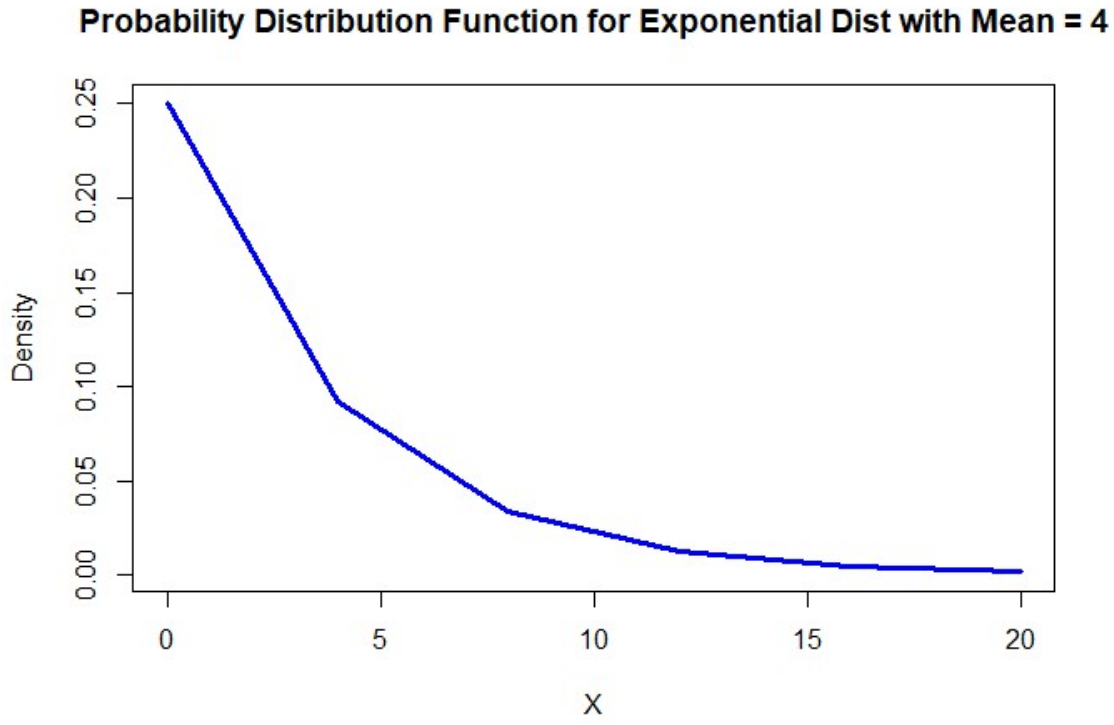
Then, add additional code that repeats the original code but uses the option `by = 4` in the `seq()` function. Why did I choose `by = .1` in the original code?

Finally, modify the code to produce a plot that shows the shape of exponential distributions with means of 1, 2, and 4 all on the same plot using different colors and line types for each distribution. Be sure to add a legend to your plot.

```r
#Get exponential probabilities - note that rate = .5 gives us mean of 2 (mean is 1/rate)
probs <- dexp(seq(0, 20, by = .1), rate = .25)

#dexp - this is an exponential density distribution and gives the height aka the mean
#rate - this gives us insight into the mean because mean = 1/rate and will also determine where the distribution will begin on the y-axis
#by - this is the step size being taken, so the number of x values being evaluated (bigger by would be bigger steps and therefore a less smooth curve)

#Plot sampling distribution
plot(seq(0, 20, by = .1), probs,
     type = 'l',
     lwd = 3,
     col = "blue",
     main = "Probability Distribution Function for Exponential Dist with Mean = 4",
     xlab = "X",
     ylab = "Density")
```

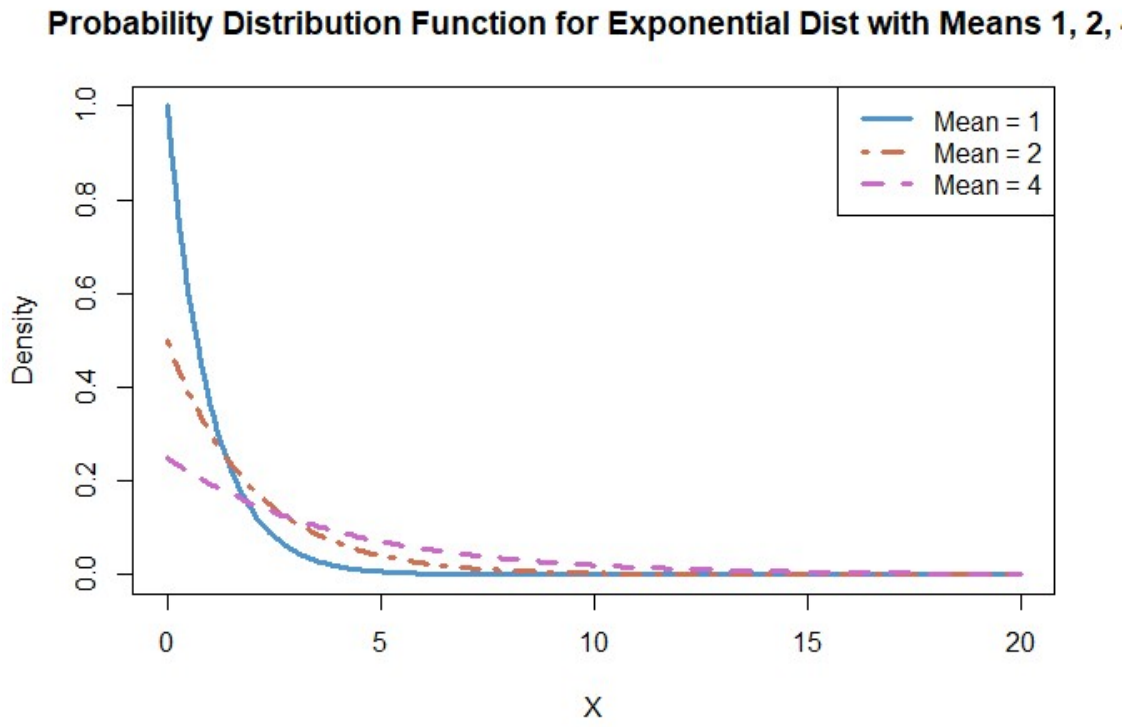



```r
#type = 'l' - this means to plot using lines
#lwd - this describes the width of the line

# by = 4 plot
probs2 <- dexp(seq(0, 20, by = 4), rate = .25) # change by = 4

plot(seq(0, 20, by = 4), probs2,
     type = 'l',
     lwd = 3,
     col = "blue",
     main = "Probability Distribution Function for Exponential Dist with Mean
= 4",
     xlab = "X",
     ylab = "Density")
```




```r
# mean = 1, rate = 1 = steel blue and lty = 1
# mean = 2, rate = .5 = salmon and lty = 4
# mean = 4, rate = .25 = orchid and lty = 2

# plotting with means 1,
x <- seq(0, 20, by = .1) # create base x to come back to to plot
rates <- c(.5, .25) # create rate for loop
colors <- c('salmon3', 'orchid3') # create colors for loop
linetypes <- c(4, 2) # create linetypes for loop

plot(x, dexp(x, rate = 1),
     type = 'l',
     lwd = 3,
     col = "steelblue3",
     lty = 1,
     main = "Probability Distribution Function for Exponential Dist with
Means 1, 2, 4",
     xlab = "X",
     ylab = "Density")

for (i in 1:length(rates)) {
  lines(x, dexp(x, rate = rates[i]),
        lwd = 3,
        col = colors[i],
        lty = linetypes[i]
```

```
        )
}

legend("topright", c("Mean = 1", "Mean = 2", "Mean = 4"), lwd = 3, col =
c("steelblue3", "salmon3", "orchid3"), lty = c(1, 4, 2))
```



You did .1 rather than 4 in the original code because the smaller value makes the curve of the density distribution appear smoother because it is plotting more values on the x axis.

(2.2) *(4 points)* Following the example in class 8, get a random sample of 20 observations from an exponential distribution with mean 4. Repeat this process 10000 times. Save your results in a matrix called `samples` with 10,000 rows and 20 columns. Display the dimension of `samples'. Show the first 3 rows ofsamples` but round the values to two decimal places.

```
# To make grading easier, please leave the following line of code in your
assignment
set.seed(230)
n_samp <- 10000

# get random sample of 20 exponential dist observation with mean = 4 aka rate
= .25
samples <- matrix(NA, nrow = n_samp, ncol = 20) # create empty matrix

for (i in 1:10000) { # does it 10,000 times
```

```
  s <- rexp(20, rate = 0.25)
  samples[i, ] <- s # fills into matrix
}

dim(samples) # ran correctly - 10,000 rows, 20 columns

## [1] 10000    20

head(round(samples, 2), 3)

##      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10] [,11] [,12] [,13]
[,14]
## [1,] 5.29 9.12 1.03 9.29 5.59 0.13 0.41 1.71 4.10  3.40  1.66  0.42  3.56
0.15
## [2,] 1.80 1.28 0.51 5.28 7.92 0.82 0.17 2.97 8.35 14.77  0.11  0.53  2.76
5.12
## [3,] 2.40 4.94 3.63 2.16 0.26 2.49 7.93 1.20 9.42  7.67  2.75  3.70  1.27
1.99
##      [,15] [,16] [,17] [,18] [,19] [,20]
## [1,]  5.87  2.38  1.60  3.76  9.61  2.19
## [2,]  1.14  7.47  4.03  4.78 10.15  4.85
## [3,]  0.30  3.50  1.30  1.83  3.90  3.51
```

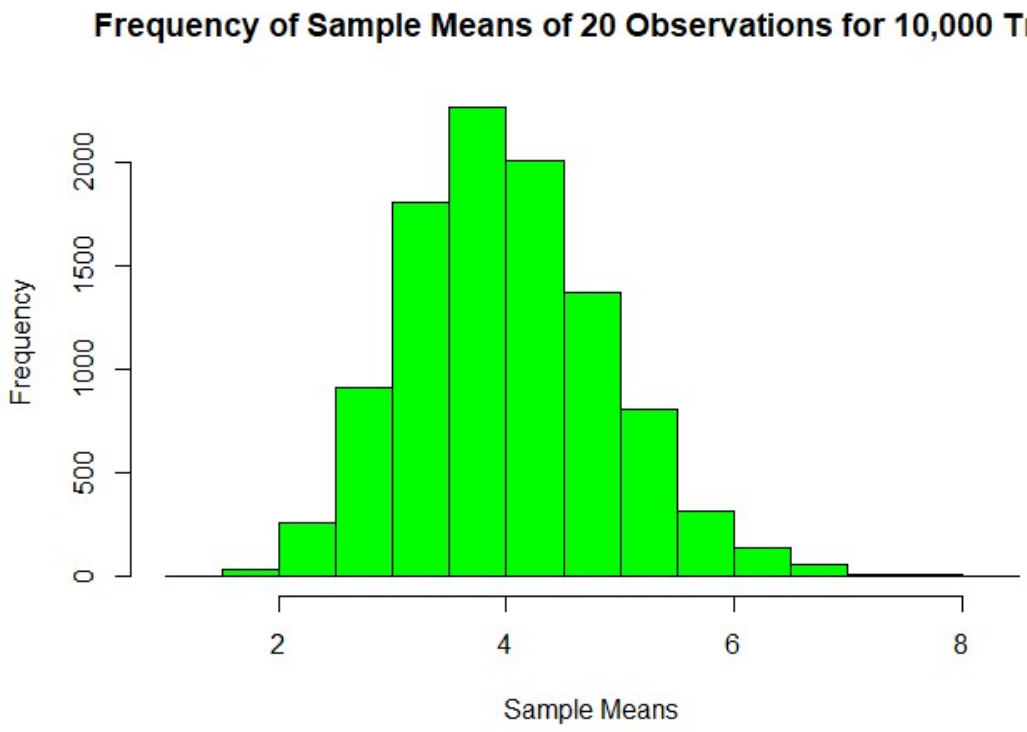
(2.3) *(4 points)* Calculate the sample mean for each sample of size 20 (i.e. calculate the mean for each row of `samples`). Repeat this process to get the sample median and the sample variance for each sample of size 20. Save these values in objects called, respectively, `smeans, smedians, svariance`.

```
smeans <- apply(samples, 1, mean)
smedians <- apply(samples, 1, median)
svariance <- apply(samples, 1, var)
```

(2.4) *(9 points)*

- Create a sample histogram of the sample means (make the bars green, make sure you label your axes and put on a clear title).
- Make a normal quantile plot of the sample means using the `qqp()` function in the `car` package. Comment on whether the CLT seems to be in effect.
- Get summary statistics OF THE SAMPLE MEANS and save this to an object called `ans1`. Using code, display only the element of `ans1` that is the sample mean, rounded to two decimal places. Is this the value you expect?
- Calculate and display the sample standard deviation of the sample means (use the function `sd()`)and display rounded to two decimal places. Then, use code to calculate the value you'd expect based on the CLT, again rounded to two decimal places. Are the two values similar?

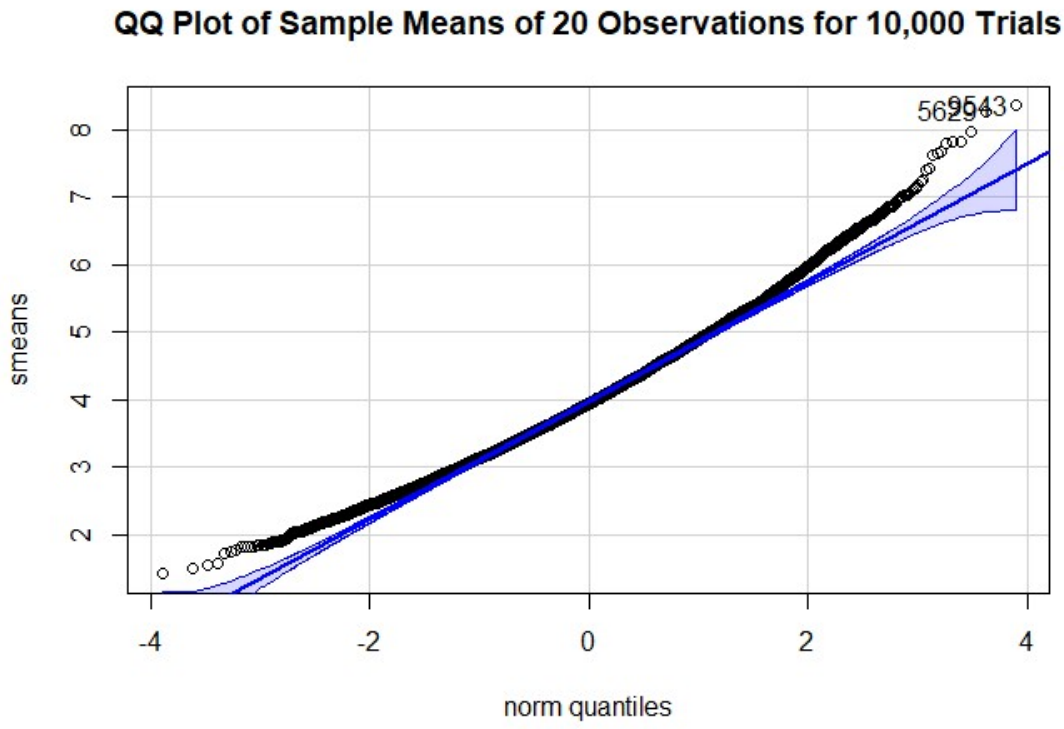
```r
# create histogram of sample means
hist(smeans,
     col = "green",
     xlab = "Sample Means",
     main = "Frequency of Sample Means of 20 Observations for 10,000 Trials")
```




```r
# make normal quantile plot
library('car') # load library needed
```

```
## Loading required package: carData
```

```r
qqp(smeans,
    main = "QQ Plot of Sample Means of 20 Observations for 10,000 Trials")
```




```
## [1] 9543 5629

# get summary statistics and display mean
ans1 <- summary(smeans)
round(ans1[4], 2)

## Mean
## 4.01

# calculate and display sd
round(sd(smeans),2)

## [1] 0.89

#sd value expected based on clt
clt_sd <- round(4/sqrt(20), 2)
clt_sd

## [1] 0.89
```

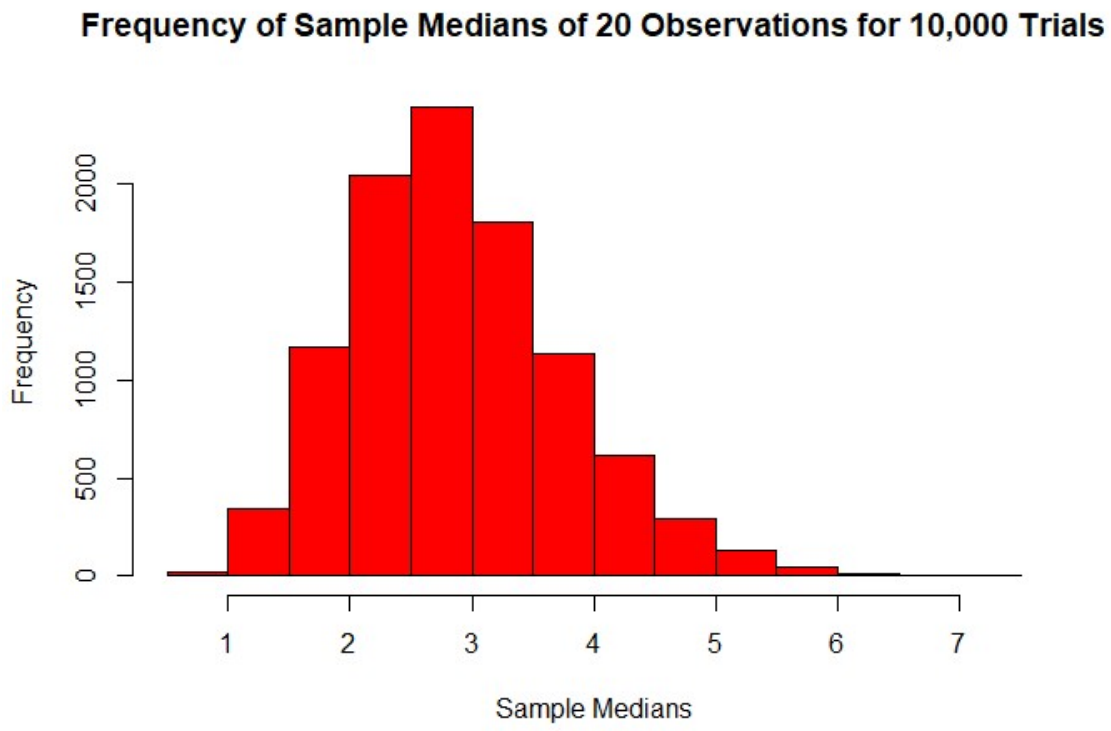
Yes CLT does seem to be in effect because the data appears to be normally distributed around 4 which we know to be the true mean. The QQ plot also suggests the data is normally distributed (although there is a slight right skewness) and thus the CLT is in effect. Yes, the mean of all the sample means is 4.006, which is very close to 4 which we know to be the true mean. Yes, the values of standard deviation from the sample and the expected standard deviation using CLT are exactly the same (0.89). You can calculate the

expected/theoretical standard deviation from CLT for an exponential distribution using mu/square root of n (4/sqrt(20)).

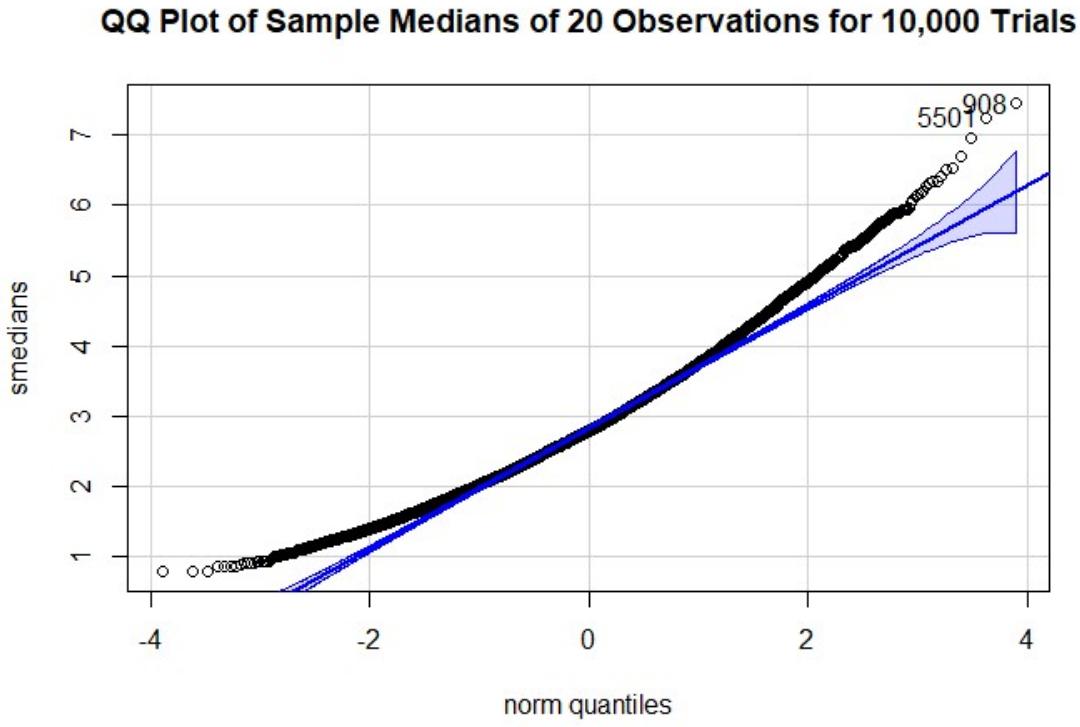
(2.5) *(7 points)*

- Create a sample histogram of the sample MEDIANS (make the bars red, make sure you label your axes and put on a clear title).
- Make a normal quantile plot of the sample medians using the `qqp()` function in the `car` package. Do the medians seem normally distributed?
- Display summary statistics OF THE SAMPLE MEDIANS. Is the median of the sample medians the value you expect?
- Calculate and display the sample standard deviation of the sample medians and display rounded to two decimal places. Is this value similar to the value you would expect? *Note that (despite what AI may tell you - I was given an incorrect answer twice), the standard deviation of the sample median is the same as the standard deviation of the sample mean*.

```
# histogram of smaple medians
hist(smedians,
     col = 'red',
     main = "Frequency of Sample Medians of 20 Observations for 10,000
Trials",
     xlab = "Sample Medians")
```




```r
# normal quantile plot of medians
qqp(smedians,
    main = "QQ Plot of Sample Medians of 20 Observations for 10,000 Trials")
```



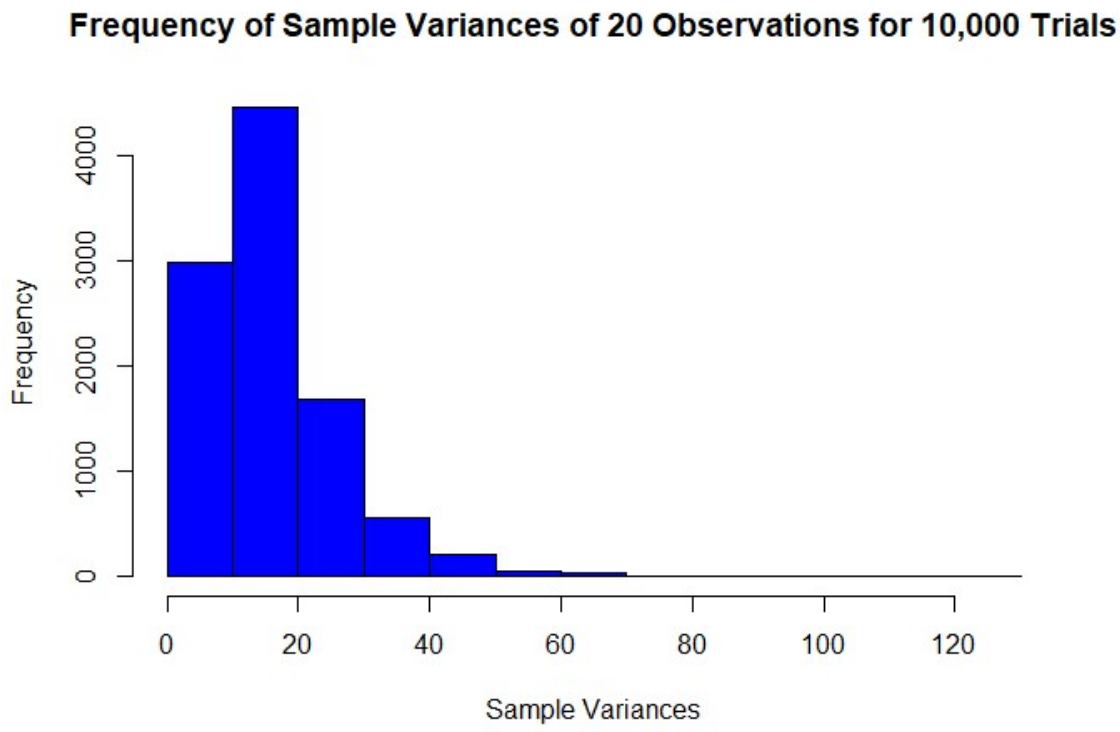

```
## [1]  908 5501

# summary statistics of sample medians
summary(smedians)

##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.7747  2.2452  2.7884  2.8817  3.4133  7.4508

median_population <- 4*log(2)

# sample standard deviation of sample median
round(sd(smedians),2)

## [1] 0.88
```

The medians is almost normally distributed, but it drifts upwards on both the right and left ends suggesting there is some right-sided skewedness. Yes, the median is as expected because the sample medians would be normally distributed around the population median (4ln2 = 2.77 for an expontential graph). Yes, the standard deviation is as expected because it is normally distributed, we would expect the variation around the mean and median to be the same so we can expect a similar value to the theoretical/expected sd as above (mean of 4 with sample size of 20 = 4/sqrt(20)).

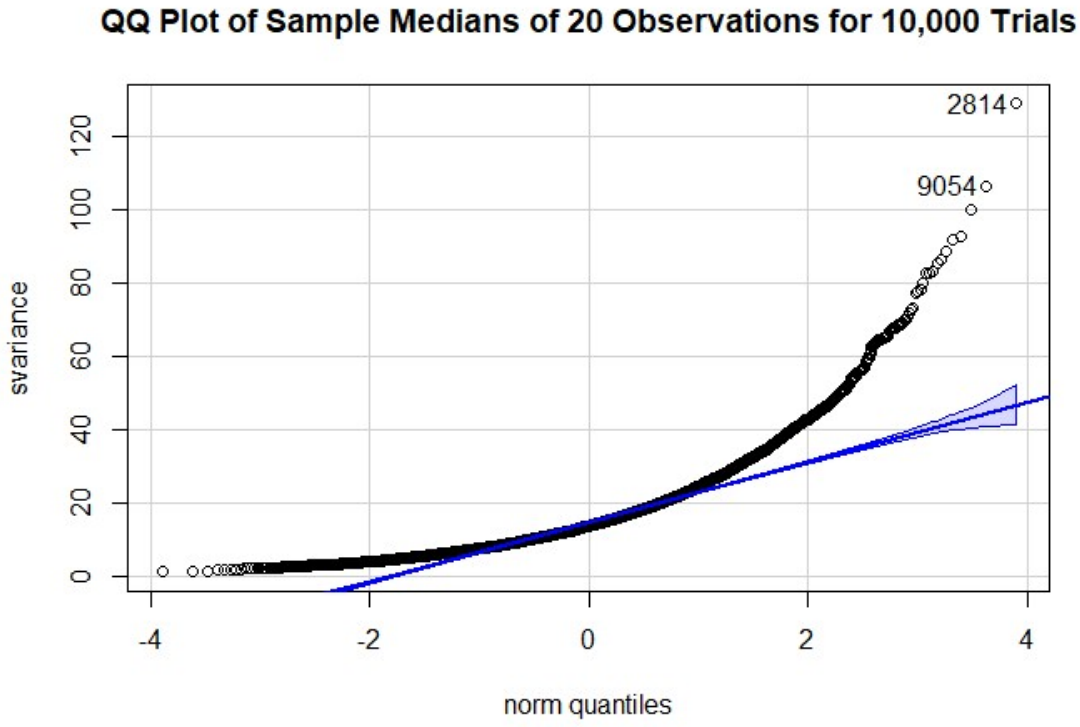
(2.6) *(6 points)*

- Create a sample histogram of the sample VARIANCES (make the bars blue, make sure you label your axes and put on a clear title).
- Make a normal quantile plot of the sample VARIANCES using the `qqp()` function in the `car` package. Do the variances seem normally distributed?
- Display summary statistics OF THE SAMPLE VARIANCES Is the mean of the sample variances the value you expect?

```
# histogram of smaple medians
hist(svariance,
     col = 'blue',
     main = "Frequency of Sample Variances of 20 Observations for 10,000
Trials",
     xlab = "Sample Variances")
```




```
# normal quantile plot of medians
qqp(svariance,
        main = "QQ Plot of Sample Medians of 20 Observations for 10,000
Trials")
```




```
## [1] 2814 9054

# summary statistics of sample medians
summary(svariance)

##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   1.108   9.149  13.697  16.087  20.230 129.035
```

These variances do not seen normally distributed. They veer highly up on the right side, suggesting it is skewed right, and also veer high on the left side, suggesting the left tail is shorter than expected. Yes, the mean of the sample variance is as expected because we would expect the population variance for this exponential graph to be 16.

(3) Iron Concentration and the Bootstrap *(50 points, 5 points each part, parts 3.5 and 3.8 count double*.

This problem examines results of a study looking at iron concentration in the the water runoff of watersheds with different mining histories above different rock types. The data is HERE. The variables are `Rock`, `Mine`, `Iron` (concentration) and `LogIron` which is the natural log of iron concentration.

(3.1) Read the data into an object called `iron`. Show the first 6 rows of the data. Show the unique values of `Rock` and `Mine`. Then modify `iron` so that it only contains data for Unmined locations. How many observations remain? How many remaining observations do you have for each rock type? Incidentally, the rock types are Limestone and Sandstone.

```r
# read data into iron
iron <-
read.csv("https://raw.githubusercontent.com/jreuning/sds230_data/refs/heads/main/IronMineRock.csv")

# show 6 first rows of data
head(iron)
```

```
##   Rock    Mine Iron logIron
## 1 Sand Unmined 0.20   -1.61
## 2 Sand Unmined 0.25   -1.39
## 3 Sand Unmined 0.04   -3.22
## 4 Sand Unmined 0.06   -2.81
## 5 Sand Unmined 1.20    0.18
## 6 Sand Unmined 0.30   -1.20
```

```r
# show unique vales of rock and mine
unique(iron$Rock)
```

```
## [1] "Sand" "Lime"
```

```r
unique(iron$Mine)
```

```
## [1] "Unmined"   "Reclaimed" "Abandoned"
```

```r
# modify iron so only contains data for unmined locations
iron <- iron[iron$Mine == "Unmined", ]
nrow(iron)
```

```
## [1] 26
```

```r
nrow(iron[iron$Rock == "Sand", ])
```

```
## [1] 13
```

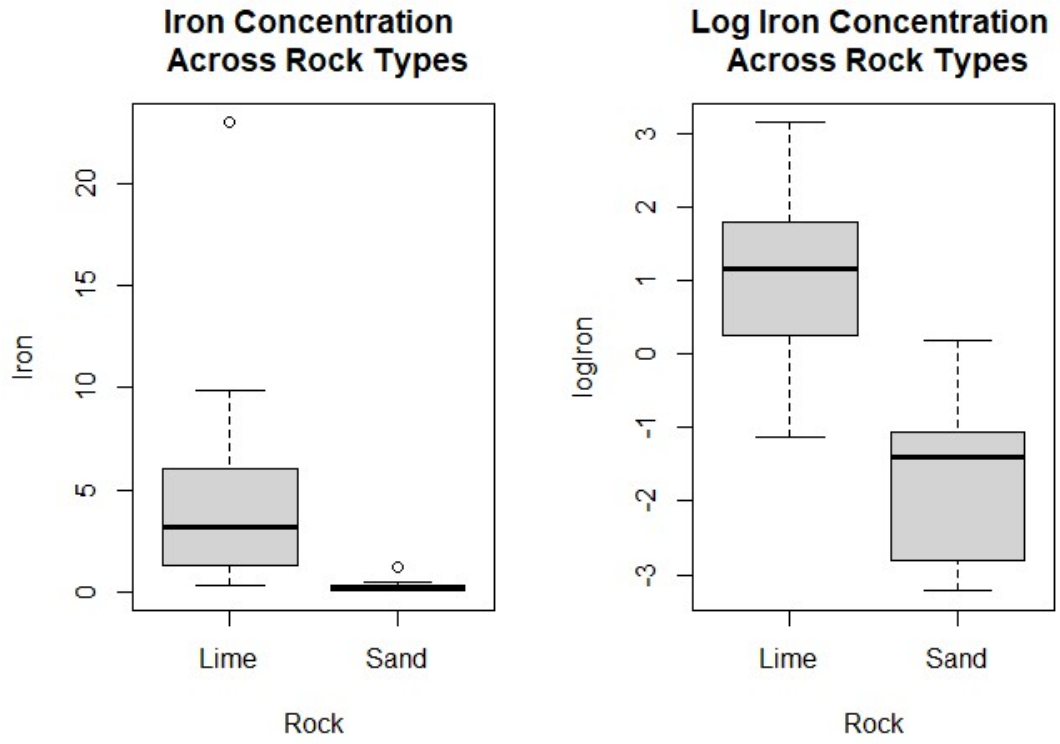
```r
nrow(iron[iron$Rock == "Lime", ])
```

```
## [1] 13
```

26 observations remain - 13 for sand and 13 for lime.

(3.2) Make side by side boxplots of iron concentration by rock type. Make this same plot for the natural log of the iron concentration. Write a sentence or two about what you observe. Which scale do you prefer?

```r
par(mfrow = c(1,2))
boxplot(Iron ~ Rock, data = iron,
        main = "Iron Concentration \n Across Rock Types")
boxplot(logIron ~ Rock, data = iron,
        main = "Log Iron Concentration \n Across Rock Types")
```



Using the raw values not in log scale, the sand observations are very condensed and small. I prefer the log values because you can appreciate the differences better and understand the full scope of data but the log does include negative values. Limestone has a higher concentration of iron compared to sandstone on average.

(3.3) Calculate summary statistics for iron concentration by rock type on the raw scale and the log scale.

```
# summary statistics for iron concentration by rock on raw scale
summary(iron$Iron[iron$Rock == "Lime"])

##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    0.32    1.30    3.20    5.17    6.00   23.00

summary(iron$Iron[iron$Rock == "Sand"])

##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.0400  0.0600  0.2500  0.2985  0.3500  1.2000

# summary statistics for iron concentration by rock on log scale
summary(iron$logIron[iron$Rock == "Lime"])

##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  -1.140   0.260   1.160   1.032   1.790   3.140

summary(iron$logIron[iron$Rock == "Sand"])
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  -3.220  -2.810  -1.390  -1.677  -1.050   0.180
```

(3.4) Calculate a two-sample t-test comparing mean log iron concentration between rock types. Save results in an object called `test1` and display the results. Use alpha = .01 (i.e. make a 99% CI). Is there evidence of a difference between rock types?

Create an object called 'test2' that has similar results for the raw iron concentration. Is there evidence of a difference between rock types?

```
# two sample t-test comparing mean log iron concentration between rock types
test1 <- t.test(iron$logIron[iron$Rock == "Lime"], iron$logIron[iron$Rock ==
"Sand"], conf.level = 0.99) # two different ways to do t test
test1

##
##  Welch Two Sample t-test
##
## data:  iron$logIron[iron$Rock == "Lime"] and iron$logIron[iron$Rock ==
"Sand"]
## t = 5.9803, df = 23.524, p-value = 3.88e-06
## alternative hypothesis: true difference in means is not equal to 0
## 99 percent confidence interval:
##  1.439543 3.977381
## sample estimates:
## mean of x mean of y
##  1.031538 -1.676923

# create object called test two that has similar results for raw iron
test2 <- t.test(Iron ~ Rock, data=iron, conf.level = 0.99) # two different
ways to do t test
test2

##
##  Welch Two Sample t-test
##
## data:  Iron by Rock
## t = 2.873, df = 12.062, p-value = 0.01395
## alternative hypothesis: true difference in means between group Lime and
group Sand is not equal to 0
## 99 percent confidence interval:
##  -0.3029436 10.0460206
## sample estimates:
## mean in group Lime mean in group Sand
##          5.1700000          0.2984615
```

Using logirons concentrations, p value = .0000388 which is < 0.01 (additionally, there is no 0 in the confidence interval) so we can reject the null hyposthesis and hypothesis that there is a difference between logiron concentrations of sandstone and limestone.

Using raw irons concentrations, p value = .0139 which is not < 0.010 (additionally, 0 is included in the confidence interval) so we must accept the null hyposthesis that there is no difference between iron concentrations of sandstone and limestone.

(3.5) Get 10,000 bootstrap samples from the data and calculate the difference in mean log iron concentration between rock types for each bootstrap sample. Save these means in an object called `diffLogIron`. In your code, use variable names that make sense.

```
# To make grading easier, please leave the following line of code in your
assignment
set.seed(230)

# i need to subset logiron values for sand and lime separately. bootstrap
from those populations independently, with replacement. for each
bootstrapping run, calculate a mean and the difference between those means.
Store each of those differences in diffLogIron
# Each bootstrap sample size will be the same size as the original data set.
I will bootstrap 10,000 times

# create empty variables needed
lime_values <- nrow(iron[iron$Rock == "Lime", ]) # create length of data to
bootstrap from
sand_values <- nrow(iron[iron$Rock == "Sand", ]) # create length of data to
bootstrap from
n_samples <- 10000 # create number of bootstrap trials
meanLogIron_lime <- rep(NA, n_samples) # empty vector to hold the means
meanLogIron_sand <- rep(NA, n_samples) # empty vector to hold the means
diffLogIron <- rep(NA, n_samples) # empty vector to hold the differences

for (i in 1:n_samples){
  l <- sample(iron$logIron[iron$Rock == "Lime"], lime_values, replace = TRUE)
#
  s <- sample(iron$logIron[iron$Rock == "Sand"], sand_values, replace = TRUE)
#
  diffLogIron[i] <- mean(l) - mean(s) # saves the mean for that run
}
```

(3.6) Calculate a 99% Bootstrap confidence interval. Show your results rounded to two decimal places.

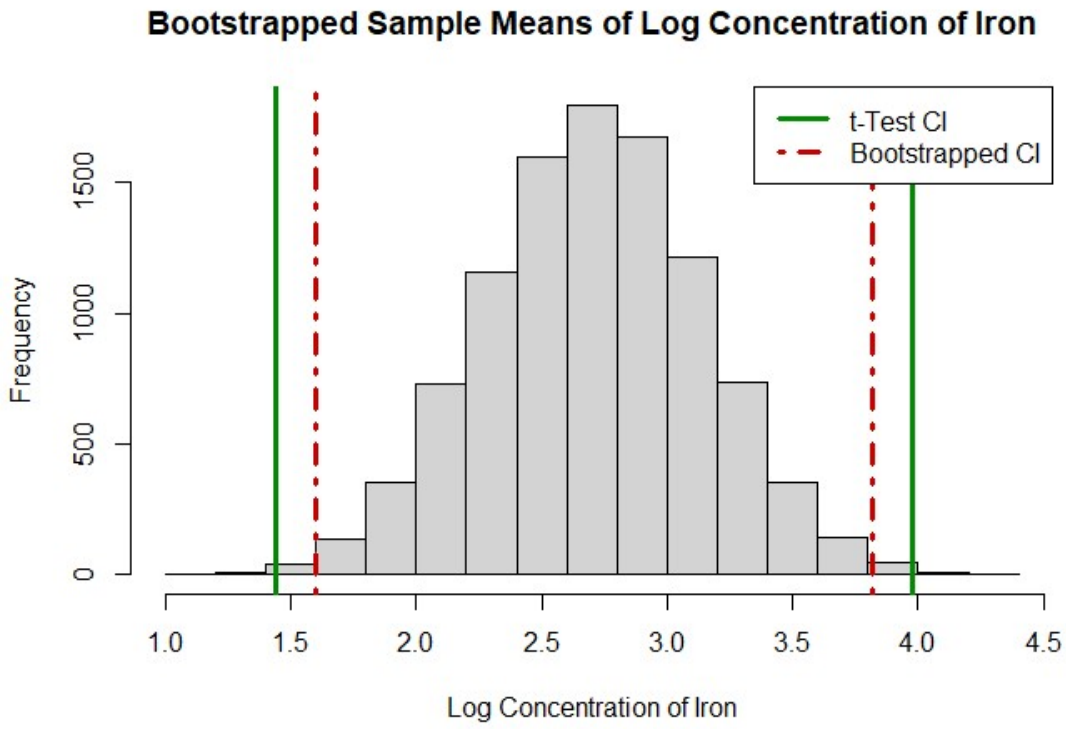
```
confidence_interval <- quantile(diffLogIron, c(0.005, 0.995))
round(confidence_interval, 2)
```

```
##  0.5% 99.5%
##  1.60  3.82
```

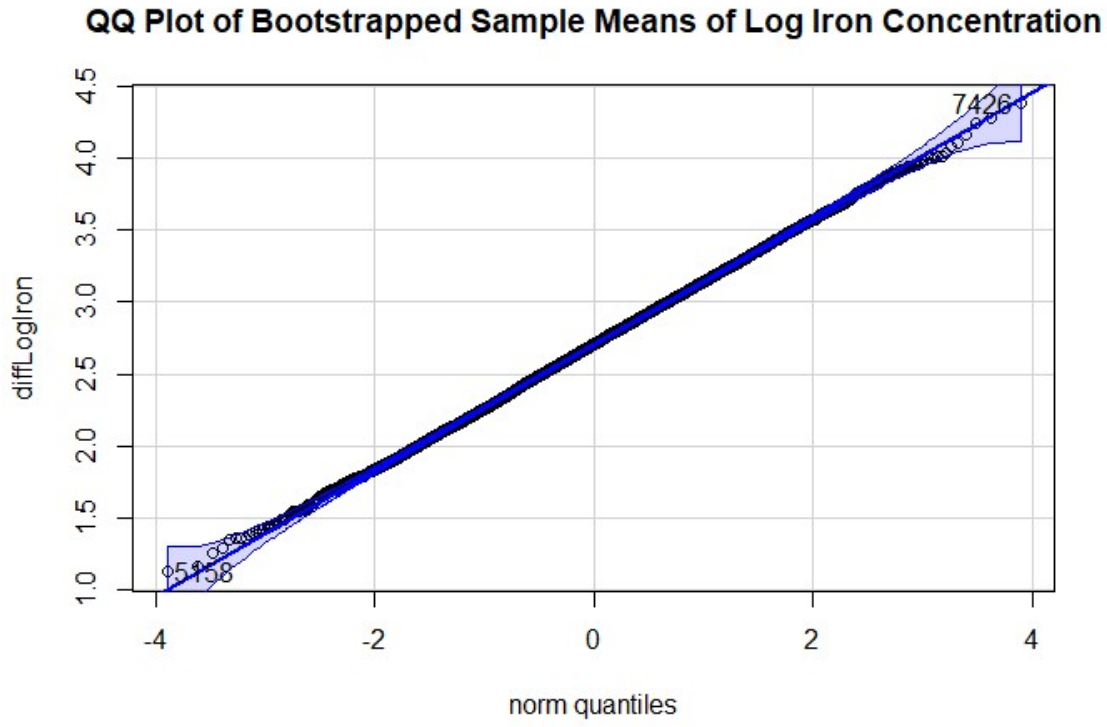
(3.7) Make a histogram of bootstrap differences in means and add vertical lines for the theoretical and bootstrapped confidence intervals (and also add a legend explaining which

lines are which type of confidence interval). Make a normal quantile plot of the bootstrapped differences. Discuss what you observe in both plots.

```
# histogram of bootstrap differences in means + vertical lines of theoretical
hist(diffLogIron,
     xlab = "Log Concentration of Iron",
     main = "Bootstrapped Sample Means of Log Concentration of Iron")

abline(v = confidence_interval, lwd = 3, col = 'red3', lty = 4) # CI of
bootstraped
abline(v = test1$conf.int, lwd = 3, col = 'green4') # CI of t test
legend("topright", c("t-Test CI", "Bootstrapped CI"), col = c("green4",
"red3"), lwd = 3, lty = c(1, 4))
```




```
# normal quantile plot of bootstrapped differences
qqp(diffLogIron,
    main = "QQ Plot of Bootstrapped Sample Means of Log Iron Concentration")
```




```
## [1] 7426 5158
```

From the histogram, we also estimate that the data is normally distributed and that 0 was not included in the confidence interval for both the bootstrapped and normal t-test and therefore we could be confident the null hypothesis would be rejected and we are seeing a difference between the mean logiron concentration levels in Limestone and Sandstone. From the quantile plot, we can see that the sample means are normally distributed. We also see that the bootstrapped confidence intervals are a little tighter compared to the confidence interval of the standard t-test, suggesting more precision in the bootstrapped data.

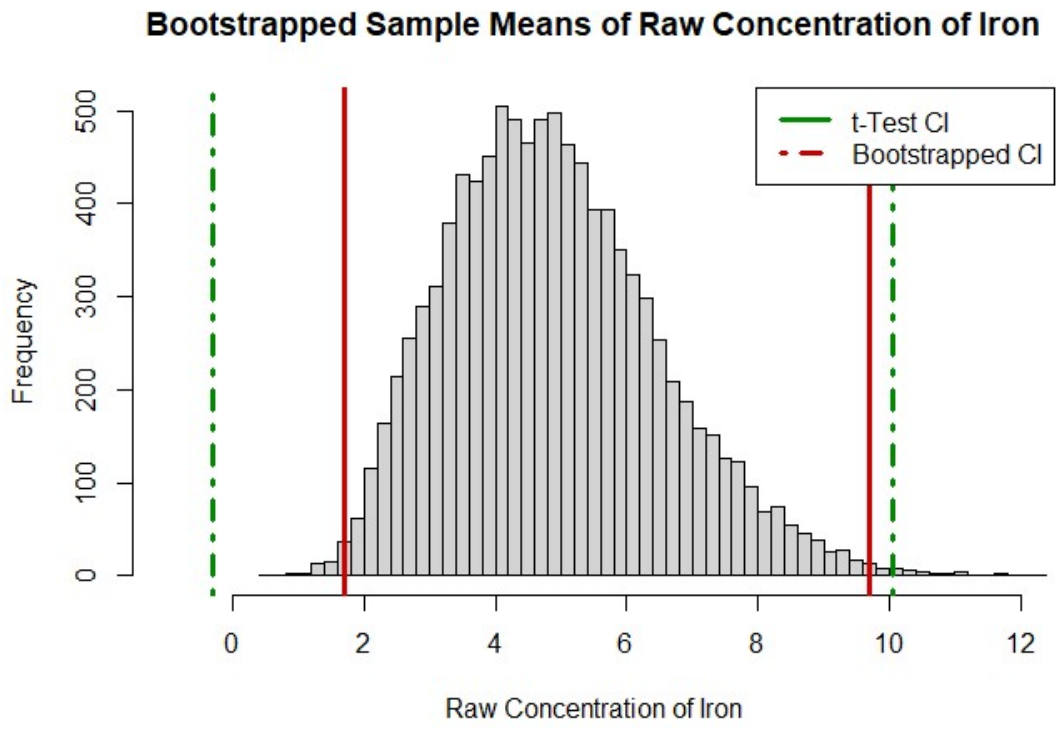
3.8) Finally, repeat 3.5 through 3.7 for iron concentrations on the raw scale. Discuss your results. Note that you'll need to add the option `xlim = c(-1, 12)` to your histogram.

```
#bootstrap

# create empty variables needed
# no need to redefine lime_values, sand_values, or n_samples
meanIron_lime <- rep(NA, n_samples) # empty vector to hold the means
meanIron_sand <- rep(NA, n_samples) # empty vector to hold the means
diffIron <- rep(NA, n_samples) # empty vector to hold the differences

for (i in 1:n_samples){
  l <- sample(iron$Iron[iron$Rock == "Lime"], lime_values, replace = TRUE) #
  s <- sample(iron$Iron[iron$Rock == "Sand"], sand_values, replace = TRUE) #
```

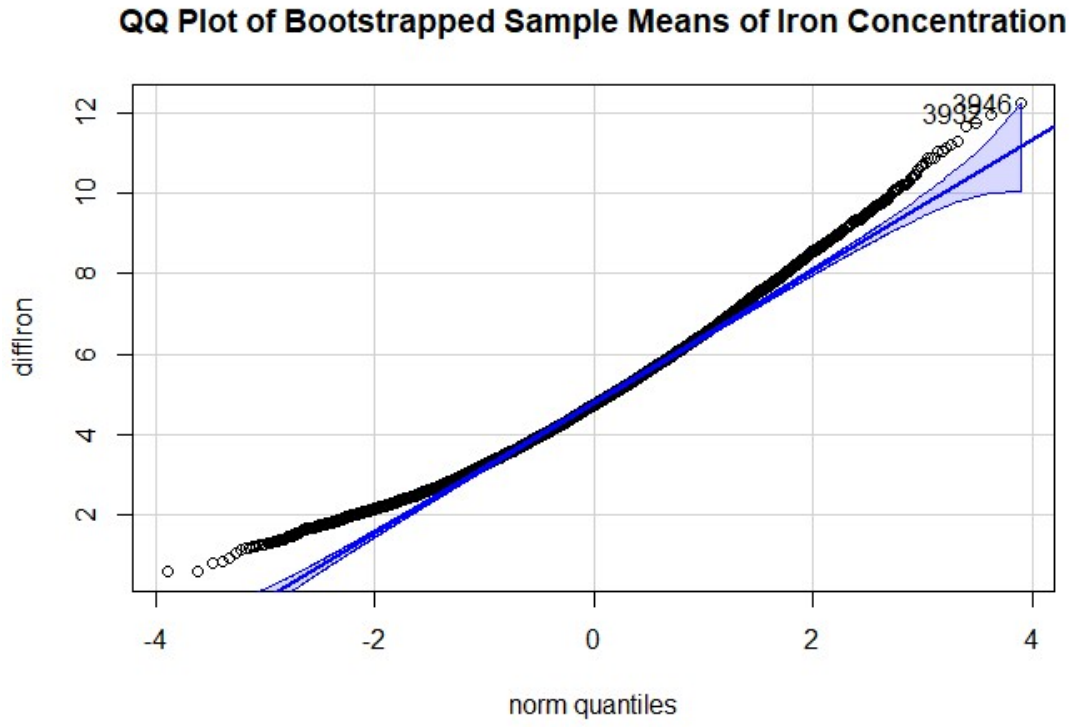
```r
  diffIron[i] <- mean(l) - mean(s) # saves the mean for that run
}

# calculate confidence interval
ci_raw <- quantile(diffIron, c(0.005, 0.995))
round(ci_raw, 2)
```

```
##  0.5% 99.5%
##  1.70  9.69
```

```r
# make histogram and qq plot
# histogram of bootstrap differences in means + vertical lines of theoretical
hist(diffIron,
     xlab = "Raw Concentration of Iron",
     main = "Bootstrapped Sample Means of Raw Concentration of Iron",
     xlim = c(-1, 12),
     breaks = 50) # adjusted to show CI

abline(v = ci_raw, lwd = 3, col = 'red3') # CI of bootstraped
abline(v = test2$conf.int, lwd = 3, col = 'green4', lty = 4) # CI of t test
legend("topright", c("t-Test CI", "Bootstrapped CI"), col = c("green4",
"red3"), lwd = 3, lty = c(1,4))
```




```r
# normal quantile plot of bootstrapped differences
qqp(diffIron,
    main = "QQ Plot of Bootstrapped Sample Means of Iron Concentration")
```




```
## [1] 3946 3932
```

The histogram of the raw concentrations using bootstrapping does not include 0 in the confidence interval so we would likely reject the null hypothesis in favor of the alternate that there is a difference in means. The traditional t-test, however, does include 0 in the confidence interval so we could not as confidently reject the null hypothesis. We also see that the confidence intervals are similar for the traditional and bootstrapped samples on the right side, but on the left, the traditional is wider, likely due to the smallness of the Sandstone values. We can also see from the QQ plot that the data is less normally distributed but still fairly considered normal. We especially see deviation on the left end because the values of sandstone are so small and concentrated on the left end of the x-axis.

4) Pseudo-coding

You are analyzing long-term water quality data from the Connecticut River. The dataset includes solute concentrations of major ions (Calcium, Magnesium, Sodium, Potassium, Chloride, Nitrate, Phosphate, and Bicarbonate) measured at locations across New England in New Hampshire, Vermont, Massachusetts, and Connecticut. Measurements have been collected from the 1950s to the present. Each row of the dataset includes the state, date, discharge, and solute name & concentration (so there are separate rows for different ions on the same date).

Write pseudocode describing the steps you would take to compare concentrations across states, examine long-term temporal trends by decade, investigate concentration–discharge (C–Q) relationships (literally concentration vs. discharge), and test whether there are statistically significant differences between groups using the tools you have learned so far.

Like with any dataset, my first steps would be to get familiar with the data. I would look at the dimensions, structure, names and head and tail of the data to understand how the data is laid out, what form the variables are in, and understand if any cleaning needed to take place. I would also see how many missing values are present across the dataset. Following any cleaning and ensuring I understand what the variables and data are representing, I would create a boxplot of the measurement of ion discharge concentration across all states and analyze the summary statistics to initially understand the data further. If any concentration values are very small, it might be advantageous to add a column that is logConcentration for each sampling. With so many variables there are many groupings and further analyses that could be made. I would then begin by making boxplots for each state for each wanted ion concentration (so, for example, this would be one boxplot graph for Vermont that has 8 boxplots, one for each major ion; so the data would be this subsetting df == to the wanted state and we would be looking at Concentration ~ Ion - if there is significant variance in the data, it might be advantageous to utilize the log(ion concentration values)). To look at long term temporal trends by decade, I would ideally create a line graph of each ion and it's concentration across the years, with each state having its own graph - although we don't technically know how to make this kind of line graph yet. To begin looking concentration-discharge relationship, we would need to create a new variable that takes the concentration and divides by discharge to create the concentration-discharge. This could be done using the mutate function, ensuring all NA's are dropped, then dividing concentration by discharge and saving it in the new created column with a meaningful name. It would be good to analyze these visually via a boxplot like the ion concentrations described above. I could analyze any significant differences between the groups utilizing a t-test. Again, due to the multitude of variables, it would likely be advantageous to conduct the t test using a for loop. I could conduct a t test between each general ion concentration (calcium vs magnesium), specific ion concentrations between states (calcium in Vermont vs calcium in New Hampshire), differences in ion concentration between years (calcium in all states in 1950 vs calcium in all states in 2026), concentration discharge calculated including all years between states (c/d of calcium in Vermont vs c/d of calcium in New Hampshire), of between years (c/d of calcium in 1950 vs c/d of calcium in 2026), or between ion (c/d of calcium vs c/d of magnesium). Utilizing a for loop would allow you to select the question you want to explore (ex: c/d of all ions compared to themselves between all states: Calcium in Vermont vs Calcium in New Hampshire, etc.) and then create vectors holding the indexing values needed. Because a t-test requires two groups and there are almost 75 years of individual data, you could also bin and combine certain years and make them into a two group cut off if wanting to look more in depth at the years(ie 1950-1980 and 1980-2026). If I wanted to be even more thorough, I could conduct bootstrapping tests as well on all grouping listed above. I could

do 10,000 trials of bootstrapping with replacement then determine the difference of each trial mean and store the differences in a variable. I would then want to plot that variable on a histogram and QQ plot to understand the distribution. I could identify the values for a 95% confidence interval and add lines on the histogram at those interval numbers. I would then use that confidence interval to determine if there is a difference between the two groups (if 0 is included, there is a difference between the groups). Again, due to the multitude of variables it would be advantageous to do this in a for loop (bootstrap samples calculate difference, confidence intervals, and plot on histogram and qq plot) and then only examine the final difference variable.

```
test <- matrix(c(1,2, 3, 4, 5),ncol=3, byrow = FALSE)
```

```
## Warning in matrix(c(1, 2, 3, 4, 5), ncol = 3, byrow = FALSE): data length
[5]
## is not a sub-multiple or multiple of the number of rows [2]
```

```
apply(test, 1, sum)
```

```
## [1] 9 7
```

```
test[1:2]
```

```
## [1] 1 2
```

```
data.frame(c(1, 2, 3, 4), ncol=5)
```

```
##   c.1..2..3..4. ncol
## 1             1    5
## 2             2    5
## 3             3    5
## 4             4    5
```